
Computer Graphics Report

Jiachen Lyu
5613244

University of Warwick

December 21, 2025

This project implements a 3D first-person shooter using C++ and DirectX 12, featuring lighting, collision detection, animation, instanced rendering, among other techniques.

1 Introduction

Movement and aiming are controlled via the keyboard and mouse. The game uses GEM models with textures loaded through the stb image library, adopts a CSV-based data-driven approach for level construction, implements multiple rendering pipelines to meet different requirements, and enhances visual presentation with a skybox and particle effects.

2 Technical Implementation

2.1 Mesh Rendering

The project implements a mesh manager that stores groups of meshes in a map and assigns them by name.

```
1 void loadmesh(Core* core, vector<vector<  
    ANIMATED_VERTEX>> vertices, vector<vector<  
    unsigned int>> indices, string meshname) {  
2     if (meshesmanager.find(meshname) !=  
        meshesmanager.end()) {  
3  
4     }  
5     else {  
6         vector<GeneralMesh*> meshes;  
7         for (int i = 0; i < vertices.size(); i++) {  
8             GeneralMesh* mesh = new GeneralMesh();  
9             mesh->init(core, vertices[i], indices[i]);  
10            meshes.push_back(mesh);  
11        }  
12        meshesmanager.insert({ meshname, meshes });  
13    }  
14  
15  
16 }
```

Listing 1: Load Meshes function

2.1.1 Data-Driven

The project uses CSV files to store map data, including the distribution of all static and dynamic models in the scene. Three separate CSV files are used for this purpose.

```
1 vector<Objectload> LoadData(vector<Objectload>*&br/>    scv1, vector<Objectload>*&br/>    scv2) {  
2     ifstream file("../Resources/loadfile1.csv");  
3  
4     string line;  
5  
6     getline(file, line);  
7     Objectload getinformation;  
8     while (getline(file, line)) {  
9         std::stringstream theline(line);  
10        std::string word;  
11  
12        getline(theline, word, ',');  
13        getinformation.type = word;  
14        getline(theline, word, ',');  
15        getinformation.name = word;  
16        getline(theline, word, ',');  
17        getinformation.position.x = stof(word);  
18        getline(theline, word, ',');  
19        getinformation.position.y = stof(word);  
20        getline(theline, word, ',');  
21        getinformation.position.z = stof(word);  
22        getline(theline, word, ',');  
23        getinformation.rotation.x = stof(word);  
24        getline(theline, word, ',');  
25        getinformation.rotation.y = stof(word);  
26        getline(theline, word, ',');  
27        getinformation.rotation.z = stof(word);  
28        getline(theline, word, ',');  
29        getinformation.location = (word);  
30        getline(theline, word, ',');  
31        getinformation.textureName = (word);  
32        getline(theline, word, ',');  
33        getinformation.iscollider = stoi(word);  
34        getline(theline, word, ',');  
35        getinformation.meshname = (word);  
36        if (getinformation.type == "static") {  
37            scv1->push_back(getinformation);  
38        }  
39        else {  
40            scv2->push_back(getinformation);
```

```

41     }
42 }
43 }
44
45 file.close();
46 return scv;
47 }

```

Listing 2: Load CSV Method

2.1.2 Mesh Rendering and Alpha Testing

During rendering, vertices are transformed from model space to clip space using the World and View-Projection matrices. In the pixel shader, textures are sampled using UV coordinates to obtain the final colour, and alpha testing is applied to discard transparent fragments.

```

1 PS_INPUT VS(VS_INPUT input)
2 {
3     PS_INPUT output;
4     output.Pos = mul(input.Pos, W);
5     output.Pos = mul(output.Pos, VP);
6     output.Normal = mul(input.Normal, W);
7     output.Tangent = mul(input.Tangent, W);
8     output.TexCoords = input.TexCoords;
9     return output;
10 }
11 // The basic vertex transformation process

```

Listing 3: From Vertex to Pixel



Figure 1: Mesh Rendering and Alpha Testing

2.1.3 Vertex Animation

In the vertex shader, grass swaying is simulated using time- and wind-based offsets, with periodic oscillations applied to the x and z axes. UV coordinates are used to weight the offset so that vertices closer to the tip receive larger offsets, and a spatial phase offset is introduced to prevent synchronized motion.

```

1 float3 GetWindEffect(float3 VertexWorldPos,
2     float GrassUV, float WindSpeed, float
3     WindStrenth, float GameTime)
4 {
5     float Timer = GameTime * WindSpeed;
6     float3 GrassWindOffset = float3(0, 0, 0);
7     float UVChange = 1 - abs(GrassUV);

```

```

8     //Due to the issue with the texture which is
9     //not a grass from bottom to top, the UV
10    weights did not achieve a satisfactory
11    result.
12    GrassWindOffset.z = UVChange * sin(cos(
13        Timer + VertexWorldPos.z)) * WindStrenth;
14    GrassWindOffset.x = UVChange * cos(sin(
15        Timer + VertexWorldPos.x)) * WindStrenth;
16    //Introduce world coordinates for pseudo-
17    randomness
18    GrassWindOffset.y = 0;
19    return GrassWindOffset;
20 }

```

Listing 4: Wind implement

$$\Delta \mathbf{p}(u, t) = (1 - |u|) w \begin{pmatrix} \cos(\sin(t + x_w)) \\ 0 \\ \sin(\cos(t + z_w)) \end{pmatrix} \quad (1)$$

2.1.4 Instancing of Meshes

instancing is used to reduce CPU workload and improve GPU utilization when rendering a large number of similar objects. An instance buffer is created on the GPU to store per-instance transformation matrices, allowing multiple instances of the same mesh to be rendered in a single draw call.



Figure 2: Instancing with Grass

2.2 Basic Camera Control System

Each frame, mouse offset updates the forward vector, from which the right vector and LookAt matrix are computed to update the camera orientation.

```

1
2     deltaX = window->mousex - lastMouseX;
3     deltaY = window->mousey - lastMouseY;
4     yaw -= deltaX * mouseSensitivity;
5     pitch -= deltaY * mouseSensitivity;
6
7     float limit = 1.5f;
8     if (pitch > limit) pitch = limit;
9     if (pitch < -limit) pitch = -limit;
10
11    forward.x = cosf(pitch) * cosf(yaw);
12    forward.y = sinf(pitch);
13    forward.z = cosf(pitch) * sinf(yaw);
14    forward.w = 0;

```

Listing 5: Convert the Angle to Forward Vector

Keyboard input determines the movement direction each frame, which is scaled by movement speed and frame time and forward vector to produce a position offset.

```

1 Vec4 groundright = Vec4(right.x, 0, right.z, 0).
  normalize();
2 Vec4 groundforawrd = Vec4(forward.x, 0, forward.
  z, 0).normalize();
3 Vec4 delta(0, 0, 0, 0);
4 if (win.keys['A']) delta += Vec4(groundright.x,
  0, groundright.z, 0);
5 if (win.keys['D']) delta -= Vec4(groundright.x,
  0, groundright.z, 0);
6 if (win.keys['W']) delta += Vec4(groundforawrd.x
  , 0, groundforawrd.z, 0);
7 if (win.keys['S']) delta -= Vec4(groundforawrd.x
  , 0, groundforawrd.z, 0);
8 //delta=delta.normalize();
9 delta *= cameramovespeed * rexdtd;
10 from = from + delta;

```

Listing 6: Move Control

2.3 Animation Control

The project implements skeletal animation with interpolated bone transforms evaluated per frame and applied in the shader.

The T-Rex employs a three-state animation system, where animations are selected based on health and collision state, with the death animation playing to completion before respawn.

```

1 void anim(float rexdtd, string status, Vec3 from)
  {
2   if (status == "death") {
3
4     if (status != Animatestatus) {
5       Animatestatus = status;
6       isdying = true;
7       dieanim = 0;
8
9     }
10    else {
11      if (enemymodelinstance.animationFinished
  () == true || dieanim > 2.8999f) {
12        health = 100.f;
13        Animatestatus = "run";
14
15        die(from);
16        isdying = false;
17        dieanim = 0;
18      }
19      else {
20
21        enemymodelinstance.update("death",
  rexdtd);
22        dieanim += rexdtd;
23      }
24    }
25  }
26  }
27
28  }
29  }
30  else {
31    Animatestatus = status;
32    enemymodelinstance.updatewithControl(
  Animatestatus, rexdtd);

```

```

  if (enemymodelinstance.animationFinished())
  {
34    enemymodelinstance.resetAnimationTime();
35    attatcktime = 0;
36  }
37
38  }
39  }

```

Listing 7: TRex Animation Control Logic

The UZI uses a five-state animation system that supports aiming, firing, and reloading, with the specific control logic implemented as shown in the code.

```

1 void anim(Vec3 from, vector<Enemies*> enemies,
  bool mouse[3], float rexdtd, Vec3 forawrd,
  bool changeR, vector<Fire*> fires) {
2   if (changeR && !ischange) {
3     ischange = true;
4     heromodelinstance.update("17 reload", rexdtd);
5     //start reload
6   }
7   if (bullet <= 0 && !ischange) {
8     ischange = true;
9     heromodelinstance.update("17 reload", rexdtd);
10    //auto reload
11  }
12  if (ischange) {
13    heromodelinstance.update("17 reload", rexdtd);
14    if (heromodelinstance.animationFinished()) {
15      heromodelinstance.resetAnimationTime();
16      ischange = false;
17      bullet = 30;
18      shot = true;
19    }
20  }
21  else {
22    if (!shot) {
23      timecount += rexdtd;
24      if (timecount > cooldown) {
25        shot = true;
26        timecount = 0.f;
27      }
28    }
29  }
30
31  if (mouse[1]) {
32    //zoom
33    if (mouse[0] && bullet > 0) {
34      if (shot) {
35        // zoom shot
36        raytest.init(from+forward*3, forawrd);
37        //plus 3 to make bullet shot front
38        of camera
39        for (int i = 0; i < enemies.size(); i
  ++){
40          float t;
41          if (enemies[i]->enemymodel.collision
  .rayAABB(raytest, t)) {
42            //hit the target or not
43            enemies[i]->health -= 5;
44            for(int i=0;i< fires.size();i++){
45              if(!fires[i]->active){
46
47                fires[i]->work(rexdtd, from +
  (forawrd * (t + 3)));
48                //particle effect
49
50              break;
51            }
52          }
53        }

```

```

54         if (enemies[i]->health <= 0&&! 116     }
enemies[i]->isdying) { 117 }
55         score += 5;
56         enemies[i]->isdying = true;
57     }
58     break;
59 }
60
61 }
62     bullet--;
63     shot = false;
64 }
65     heromodelinstance.update("13 zoom fire",
rexdtd);
66
67     if (heromodelinstance.animationFinished()
) {
68         heromodelinstance.resetAnimationTime();
69         shot = true;
70     }
71 }
72 else {
73     heromodelinstance.updatewithControl("11
zoom idle", rexdtd);
74 }
75
76 }
77 }
78 else if (mouse[0] && bullet > 0) {
79     //normal shot
80     if (shot) {
81         raytest.init(from + forward * 3, forawrd
);
82         for (int i = 0; i < enemies.size(); i++)
{
83             float t;
84             if (enemies[i]->enemymodel.collison.
rayAABB(raytest, t)) {
85                 enemies[i]->health -= 5;
86                 for (int i = 0; i < fires.size(); i
++) {
87                     if (!fires[i]->active) {
88
89                         fires[i]->work(rexdtd, from + (
forawrd * (t+3)));
90
91
92                     break;
93                 }
94             }
95             if (enemies[i]->health <= 0 && !
enemies[i]->isdying) {
96                 score += 5;
97                 enemies[i]->isdying = true;
98             }
99             break;
100         }
101     }
102 }
103     bullet--;
104     shot = false;
105 }
106     heromodelinstance.update("08 fire", rexdtd);
107     if (heromodelinstance.animationFinished())
{
108         heromodelinstance.resetAnimationTime();
109         shot = true;
110     }
111 }
112 else {
113     heromodelinstance.updatewithControl("04
idle", rexdtd);
114     //idle
115 }

```

Listing 8: UZI Animation Control Logic

2.4 Collision System

The project employs AABB and ray-based collision detection. Enemy collision boxes are generated by computing the minimum bounding box from vertex extrema in model space and transforming it into world space. AABB collisions are detected by testing overlap along each axis, where separation on any axis indicates no collision.

```

1 bool AABBtest(const Vec3& minB, const Vec3& maxB
)
2 {
3
4     if (realmaxpoint.x < minB.x || realminpoint.x
> maxB.x)
5         return false;
6     if (realmaxpoint.y < minB.y || realminpoint.y
> maxB.y)
7         return false;
8     if (realmaxpoint.z < minB.z || realminpoint.z
> maxB.z)
9         return false;
10
11     return true;
12 }

```

Listing 9: Collision Test

During rotation, the eight corner vertices are constructed and the minimum bounding box is recomputed.

```

1 void update(Vec3 position, Matrix scv) {
2     //rotation need calculate the 8 corner point
and find the new min and max point
3     vector<Vec3>corner;
4     corner.push_back(Vec3(minpoint.x, minpoint.y,
minpoint.z));
5     corner.push_back(Vec3(minpoint.x, minpoint.y,
maxpoint.z));
6     corner.push_back(Vec3(minpoint.x, maxpoint.y,
minpoint.z));
7     corner.push_back(Vec3(minpoint.x, maxpoint.y,
maxpoint.z));
8     corner.push_back(Vec3(maxpoint.x, minpoint.y,
minpoint.z));
9     corner.push_back(Vec3(maxpoint.x, minpoint.y,
maxpoint.z));
10    corner.push_back(Vec3(maxpoint.x, maxpoint.y,
minpoint.z));
11    corner.push_back(Vec3(maxpoint.x, maxpoint.y,
maxpoint.z));
12    for (int i = 0; i < 8; i++) {
13        corner[i] = scv.MulVec3(scv, corner[i]);
14    }
15    realminpoint = Vec3(11111, 11111, 11111);
16    realmaxpoint = Vec3(-11111, -11111, -11111);
17    for (int i = 0; i < 8; i++) {
18        if (realmaxpoint.x < corner[i].x) {
19            realmaxpoint.x = corner[i].x;
20        }
21        if (realmaxpoint.y < corner[i].y) {
22            realmaxpoint.y = corner[i].y;
23        }
24        if (realmaxpoint.z < corner[i].z) {

```

```

25     realmaxpoint.z = corner[i].z;
26 }
27 if (realminpoint.x > corner[i].x) {
28     realminpoint.x = corner[i].x;
29 }
30 if (realminpoint.y > corner[i].y) {
31     realminpoint.y = corner[i].y;
32 }
33 if (realminpoint.z > corner[i].z) {
34     realminpoint.z = corner[i].z;
35 }
36 }
37 realminpoint += position;
38 realmaxpoint += position;
39
40
41 }

```

Listing 10: rotation collider box

Ray-AABB intersection is determined by comparing per-axis entry and exit times; an intersection occurs only if the exit time is greater than the entry time, and objects behind the ray are ignored

```

1 bool rayAABB(const Ray& r, float& t)
2 {
3     Vec3 s = (realminpoint - r.o) * r.invdir;
4     Vec3 l = (realmaxpoint - r.o) * r.invdir;
5     //calculate the time of input and output in
6     //xyz directions
7     Vec3 s1 = Min(s, l);
8     Vec3 l1 = Max(s, l);
9     //if the direction is negative, swap the value
10    float ts = max(s1.x, max(s1.y, s1.z));
11    //the max number in min, the real enter time
12    float tl = min(l1.x, min(l1.y, l1.z));
13    //the min number in max, the real leave time
14    t = min(ts, tl);
15
16    if (t1 < 0.0f)
17        return false;
18    //direction is negative
19
20
21    return (ts < tl);
22
23
24 }

```

Listing 11: Ray Test

2.5 Normal Mapping, Lighting, and Textures

In the pixel shader, textures are sampled using UV coordinates to obtain the final pixel colour. Textures are stored in the TextureManager class using a map structure and are accessed by name. Textures for GEM models are loaded during the model loading stage, while textures for custom geometry are loaded in the main function and assigned after initialization.

```

1 void load(Core* core, vector<string> name, vector
2     <string>NHname, string texturename)
3 {
4     if (textures.find(texturename) != textures.end
5         ()) {
6
7     }
8 }

```

```

6 else {
7     vector <Texture*> scv;
8     vector <Texture*> scv1;
9     for (int i = 0; i < name.size(); i++) {
10
11         Texture* texture = new Texture;
12
13         texture->load(core, name[i]);
14         scv.push_back(texture);
15
16
17         Texture* texture1 = new Texture;
18
19         texture1->load(core, NHname[i]);
20         scv1.push_back(texture1);
21
22
23     }
24     textures.insert({ texturename, scv });
25     NHtextures.insert({ texturename, scv1 });
26 }
27
28
29 }

```

Listing 12: Load Textures Function

Lighting is implemented by computing the angle between the surface normal and the vector from the surface to the light source to determine the actual light intensity, with an additional ambient term to prevent the scene from appearing too dark.

```

1 float3 ComputeDirectionalLight(Light L, float3
2     normal )
3 {
4     float3 lightVec = -normalize(L.Direction); //
5     //calculate the vector from surface to light
6     float ndotl = max(dot(lightVec, normal), 0.0
7     f); //Lambert Cosine Law, calculate the
8     //strength of light
9     float3 lightStrength = L.Strength * ndotl; //
10    mix
11
12    return lightStrength;
13 }

```

Listing 13: Light Function

$$L_{dir} = S \cdot \max\left(\mathbf{n} \cdot \left(-\frac{\mathbf{d}}{\|\mathbf{d}\|}\right), 0\right) \quad (2)$$

Normal mapping is applied by sampling the normal map in UV space and transforming it to world space using the TBN matrix for lighting calculations.



Figure 3: Lighting Effect

```

1 float4 PS(PS_INPUT input) : SV_Target0
2 {
3
4     float4 colour = tex.Sample(samplerLinear,
5         input.TexCoords);
6     if (colour.a < 0.5f)
7         discard;
8     float3 normalTS = NHtex.Sample(samplerLinear
9         , input.TexCoords).xyz;
10    normalTS = normalize(normalTS * 2.0f - 1.0f)
11    ; // turn Vec to normal
12
13    float3 N = normalize(input.Normal);
14    float3 T = normalize(input.Tangent);
15    T = normalize(T - dot(T, N) * N);
16    float3 B = cross(N, T);
17    float3x3 TBN = float3x3(T, B, N);
18    float3 normalWS = normalize(mul(normalTS,
19        TBN)); //turn to world space normal
20
21    Light L;
22    L.Strength = Strength;
23    L.Direction = normalize(Direction);
24
25    float4 ambient = gAmbientLight ; //ambient
26    light
27
28    float4 directLight = ComputeLighting(L,
29        input.PosW, normalWS);
30    float3 lighting = ambient.rgb + directLight.
31    rgb;
32    float3 finalColor = colour.rgb * lighting;
33    return float4(finalColor, colour.a);
34 }

```

Listing 14: Pixel Shader for Lighting

2.6 Other Effects

2.6.1 Skybox

The skybox is rendered using a spherical mesh and texture, follows the camera position, and participates in depth testing without writing to the depth buffer to remain a background element.



Figure 4: skyboy

2.6.2 UI

UI elements are rendered last without depth testing using textured planes positioned directly in screen space.

```

1 PS_INPUT VS(VS_INPUT input)
2 {
3     PS_INPUT output;
4
5     output.Pos = float4((input.Pos.x-960)/960, (
6         input.Pos.y-540)/540, 0.0f, 1.0f);
7     //calculate to [-1,1]
8
9     output.Normal = input.Normal;
10    output.Tangent = input.Tangent;
11
12    output.TexCoords = input.TexCoords;
13
14    return output;
15 }

```

Listing 15: Vertex Shader for UI

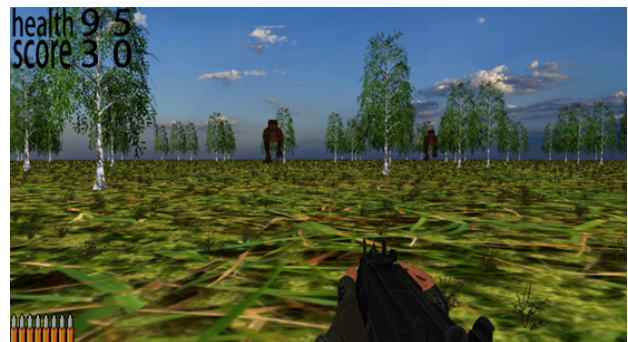


Figure 5: UI

2.6.3 Particles Effect

When a ray hits an object, a spark-like particle effect is triggered. Particle directions and speeds are generated using pseudo-random values, and their positions evolve over time. The particles are rendered on a billboard, with their intensity determined based on distance.



Figure 6: Particle Effect

```

1 float3 FireworkParticles( float3 worldPos, float3
2     fireworkPos, float time, float3 baseColor
3 )
4 {
5     float seed = hash1(dot(fireworkPos, float3
6         (12.9898, 78.233, 45.164)));
7     //random seed
8     float3 color = float3(0, 0, 0);
9 }

```

```

8  const int NUM_PARTICLES = 109;
9
10 for (int i = 0; i < NUM_PARTICLES; i++)
11 {
12     float speed = lerp(3.0, 6.0, hash1(i *
13 seed* 31.35));
14     //get random speed form 3 to 6
15     float3 dir = normalize(float3(
16         hash1(i*seed * 33.95)*2-1 ,
17         hash1(i*seed * 7.1)*2-1 ,
18         hash1(i*seed * 25.12)*2-1
19     ));
20     //random direction
21     float3 p = fireworkPos + dir * speed *
22 time;
23     //now position of particle
24     float dist = length(worldPos - p);
25     float size = 0.1;
26     float fade = saturate(1.0 - time);
27     float intensity = smoothstep(size, 0.0,
28 dist) * fade;
29     //calculate color based on distance
30     //the dis more close to zero more light
31     color += baseColor * intensity;
32 }
33
34 return color;
35 }

```

Listing 16: Particle Effect implement

$$v_i = v_{\min} + (v_{\max} - v_{\min}) \cdot h(i \cdot s \cdot k_v) \quad (3)$$

$$\mathbf{d}_i = \text{normalize} \begin{pmatrix} 2h(i \cdot s \cdot k_1) - 1 \\ 2h(i \cdot s \cdot k_2) - 1 \\ 2h(i \cdot s \cdot k_3) - 1 \end{pmatrix} \quad (4)$$

3 Performance Analysis

Tree models are used to compare frame rates between standard rendering and instanced rendering at the same object counts, in order to evaluate the impact of CPU bottlenecks on performance.



Figure 7: test scene

Table 1: Performance Test Results Under Different Conditions

Render Method	Tree Number	FPS
Normal	100	337
Normal	200	195
Normal	300	133
Normal	400	101
Normal	500	81
Normal	600	61
Normal	800	47
Normal	1000	39
Instancing	100	2747
Instancing	200	1653
Instancing	300	1389
Instancing	400	940
Instancing	500	745
Instancing	600	653
Instancing	800	514
Instancing	1000	412

The table shows that when rendering a large number of similar objects, CPU bottlenecks have a significant impact on frame rate.

4 Limitations

The project suffers from a tightly coupled structure due to the lack of an initial architectural design, making further extension difficult. In addition, the UZI model shows texture jittering likely caused by z-fighting between closely positioned meshes.

5 Future Work

The project requires a clearer overall structure, including consistent file path conventions to reduce errors from scattered assets. Similar object classes would benefit from a unified inheritance hierarchy to improve maintainability, and the texture system should avoid grouped storage to prevent redundancy by storing textures once and referencing them by index.

6 Conclusion

This project implements a first-person shooter using C++ and DirectX 12, featuring animation, lighting with normal mapping, instancing, particle effects, and other related rendering techniques. Instancing is used to efficiently render large numbers of objects, while particle effects and a skybox enhance the visual quality. Overall, the project meets its intended objectives.

Project Repository

The source code for this project is available on GitHub:
<https://github.com/AODINGLVP/newworld.git>